CLAUDE CERTIFIED ARCHITECT

What to know for the exam.

Full course: certmasterclass.com/certification/claude-certified-architect-foundations

By Jacob Bushong

Scan to open the full course.

EXAM FORMAT

- 60 scenario-based questions
- 120 minutes
- 720 / 1000 to pass
- Proctored via Pearson VUE

Domain 1: Agentic Architecture & Orchestration

Exam weight: 27%

Section 1A - Agentic Loops and Core API

What Makes a System Agentic *(Lessons 1 and 2)*

The line between agents, workflows, and chatbots, and how tool use turns a language model into a system that acts in the world.

- An agentic system is one where the model controls flow at inference time. Workflows fix flow at design time; chatbots only respond to messages.
- Agency requires four pillars working as a loop: perception, selection, execution, iteration.
- Tool use closes the action-environment gap. The model emits a structured tool call, the runtime executes it, and the result returns to the model as an observation.
- Write tool descriptions a non-engineer could act on. Description quality, not code quality, drives how reliably the model picks the right tool.
- Reach for agentic architecture only when the path to the goal can't be pre-scripted. Higher autonomy buys higher capability and a larger risk surface.

The Agentic Loop *(Lessons 3 and 4)*

The four-phase cycle every agent runs, and the controls that keep it from running forever.

- The loop runs four phases continuously: Perception reads context; Reasoning chooses the next action; Action invokes a tool; Observation returns the result to context.

- The context window is the agent's ephemeral working memory between turns. State that needs to persist across runs has to live elsewhere.
- Define an explicit termination condition before deploying any agent. Without one, the loop has no clean exit.
- Use turn budgets in the runtime layer to cap execution and prevent infinite loops driven by ambiguous goals or repeating tool errors.
- Escalate to a human when the agent stops making progress. Compare successive observations to detect spinning, and route out rather than cycling indefinitely.

Agents vs. Workflows *(Lessons 5 and 6)*

How to choose between an agent and a workflow for a given task, and why the answer is workflow more often than people expect.

- Workflows beat agents when the task is predictable, the output shape is known up front, and every step can be authored at design time.
- Four core workflow patterns cover most predictable work: chaining, routing, parallelization, and orchestrator-subagent.
- Reach for an agent only when the path to the goal genuinely can't be pre-scripted. Tool use alone does not make a system agentic.
- Evaluate every architecture choice on four axes: cost, auditability, reliability, adaptability. Each one trades off against the others.
- Hybrid architectures pair workflow structure with targeted agent behavior at the steps where adaptability actually adds value.

Section 1B - Task Decomposition and Planning

Task Analysis and Decomposition Strategies *(Lessons 7 and 8)*

How to break complex goals into structured subtasks, and the three patterns that organize agentic workflows.

- Decompose goals before assigning them to agents. Monolithic tasks fail completely on any error; decomposed tasks allow targeted retry of only the step that broke.
- A good subtask has four properties: single responsibility, defined inputs, bounded scope, and clean structured output the orchestrator can route without parsing free-form text.
- Handoff points are the interfaces between subtasks. Pass only what the next step needs, use explicit schemas, and document what is not passed to prevent silent false assumptions.
- Hierarchical decomposition builds a goal tree where each orchestration level handles only its tier of complexity. Flatten layers whenever two adjacent levels would use the same agent type.
- Three anti-patterns account for most agentic pipeline failures: over-decomposition wastes coordination budget, under-decomposition blocks selective retry, and false parallelism treats dependent steps as independent and causes data hazards.

Sequential vs. Parallel Execution Patterns *(Lessons 9 and 10)*

When to enforce step ordering and when to fan out, using dependency graphs to choose correctly between sequential and parallel execution.

- Sequential execution is correct when data dependencies are real, step B literally needs step A's output, and forcing order does not waste latency you could otherwise recover.
- Draw a dependency graph before committing to a sequential design. Steps with no incoming edges can often run in parallel, and many pipelines are more sequential than they need to be.
- Audit every pipeline for artificial dependencies, steps that are sequential only by convention rather than data necessity. Removing them is typically the highest-leverage latency reduction available.
- Fan-out dispatches independent branches simultaneously from a single control point. Run the independence test first: branches must not share mutable state and must produce the same result regardless of execution order.
- Fan-in collects results, handles partial failures, reorders outputs if sequence matters, and merges before the pipeline continues. Never silently drop failed branches; define an explicit policy for abort, partial-proceed, or per-branch retry.

Dynamic Planning, Replanning, and Ambiguity Handling *(Lessons 11 and 12)*

How agents adapt when execution diverges from the original plan, and how to handle goals that were never fully specified to begin with.

- Static plans are authored at design time and are predictable and auditable. Dynamic plans are generated or revised at inference time and enable adaptability but require stronger monitoring and explicit fallback paths.
- Three event types trigger replanning: a tool call fails or returns nothing useful, a step returns valid output that contradicts the next step's assumption, or the environment shifts and earlier plan steps are no longer valid.
- Not every surprise warrants a full replanning cycle. Replan when a core assumption has been invalidated; continue when the deviation is minor and the original path is still recoverable.
- Bound every replanning loop with a maximum iteration limit and a goal constraint check that validates each revised plan against original intent before execution resumes, preventing goal drift.
- For ambiguous goals, clarify upfront when actions are irreversible, high-cost, or scope-uncertain. Assume and proceed when actions are low-stakes and reversible. Always document assumptions in agent output so humans can audit reasoning without rerunning the task.

Section 1C - Multi-Agent Orchestration

Orchestrator-Subagent Model *(Lessons 13 and 14)*

How the orchestrator directs a multi-agent system, and why subagent isolation is not a limitation but a design property.

- The orchestrator performs three functions in sequence: decompose the goal into bounded subtasks, assign each subtask to the right subagent, and aggregate the results. All three must be explicitly designed.

- Error handling belongs in the orchestrator, not in subagents. When a subagent fails, the orchestrator decides whether to retry, substitute, or escalate to a human.
- Subagents operate in their own context window. Nothing from the orchestrator's history is visible by default. Every piece of information the subagent needs must be passed explicitly at assignment time.
- Grant subagents least-privilege authority: exactly the tools and data the subtask requires, and nothing more. Subagents should not take actions outside their assigned scope without orchestrator approval.
- Write subagent instructions with narrow scope, a precise output format, and explicit success criteria. Vague instructions produce inconsistent outputs that break aggregation downstream.

Multi-Agent Topology Patterns *(Lessons 15 and 16)*

Three structural patterns for wiring agents together, each optimizing for different priorities and failing in different ways.

- Hub-and-spoke routes all tasks through a central orchestrator. Spokes never communicate directly. The hub provides centralized state, a single audit trail, and straightforward failure handling.
- Hub-and-spoke's centralization is also its weakness: the hub is a single point of failure, a throughput bottleneck, and accumulates context bloat as active spokes grow.
- Pipeline topology runs agents in strict sequence. Each stage produces a clean, testable artifact for the next. Total latency equals the sum of all stage latencies, and a failed mid-stage agent blocks every downstream stage.
- Peer-to-peer removes the central coordinator entirely, reducing latency and eliminating the single-point-of-failure risk. The cost is distributed state that is hard to audit and emergent behavior that is hard to predict.
- The evaluator-optimizer pattern pairs a generator agent with a critic agent in a loop. A maximum iteration count is a required design element, not optional, to guarantee termination.

Agent Communication and Handoffs *(Lessons 17 and 18)*

Structured handoff schemas and verified protocols are what separate reliable multi-agent systems from brittle ones.

- Every handoff message needs four elements: task description, relevant context, output format specification, and explicit constraints. Anything the receiver cannot act without must be included; everything else should be excluded.
- Under-specified handoffs leave the receiver guessing. Over-specified handoffs inflate token cost and add noise. The target is minimum necessary context, confirmed sufficient by a simulated receiver test.
- Tag every schema with a version field. Receivers must handle missing optional fields gracefully, and new required fields need a migration path for older senders. Forward compatibility keeps the system stable during incremental updates.
- A completed handoff requires verification, not just delivery. The receiver acknowledges receipt and confirms it has the context needed to proceed before the sender relinquishes control.
- Subagents must return structured error payloads, not silent nulls. Swallowed errors and undifferentiated error codes both prevent the orchestrator from choosing the right recovery response.

State and Session Management *(Lessons 19 and 20)*

Where state lives and how it survives interruptions are foundational design decisions in every agentic system.

- In-context state offers zero retrieval latency but is bounded by the context window and lost when the session ends. External memory persists across sessions and supports key, filter, and semantic queries, but requires a tool call on every access.
- Classify state before deciding where it lives: hot state actively referenced this turn stays in context, cold state that must outlast the session goes external, reconstructable state often needs no storage at all.
- Checkpoint at phase boundaries, not after every tool call. Each checkpoint must record the phase marker, step outputs, task inputs, and a timestamp before any irreversible action in the next phase begins.
- Version every checkpoint with an environment hash. On resume, compare the current environment against the saved version. Flag conflicts rather than silently proceeding on stale state.
- Design for graceful degradation at every phase boundary. An agent that requires complete state will halt entirely on any gap. Partial state should trigger reduced-capability continuation, not a full restart.

Section 1D - Hooks and Programmatic Enforcement

Error Classification in Agentic Systems *(Lessons 21 and 22)*

How to classify agentic failures by origin and why that classification determines every recovery decision.

- Three error categories cover all agentic failures: tool errors originate in external APIs or services, reasoning errors originate inside the model, and environment errors originate in infrastructure below both.
- Classify before choosing a recovery strategy. Applying a retry loop to a reasoning error just reproduces the same wrong answer because the model's context has not changed.
- Tool errors split on retriability: transient ones (timeouts, rate limits, temporary unavailability) can succeed on retry; permanent ones (invalid input, permission denied, not found) require fallback or escalation.
- Silent failures are the hardest detection case because no error code is raised. Output validation gates must inspect schema, ranges, and completeness at every stage boundary.
- Use state verification after write operations and sanity checks on output length and entity coverage to catch errors that pass structural validation but are semantically wrong.

Fallback and Retry Strategies *(Lessons 23 and 24)*

When to retry, when to abort, and how to build ordered fallback chains that degrade gracefully instead of silently.

- Every retry decision starts with one classification question: will this error resolve if I try again? Transient errors are retriable; persistent errors should abort immediately and trigger escalation.
- Apply exponential backoff with jitter to transient errors. The doubling interval gives downstream systems recovery time, and jitter prevents synchronized retry storms across concurrent clients.
- Set a retry budget before deploying any agent. Uncapped retries accumulate token cost and pipeline latency with no guarantee of eventual success; abort cleanly when the budget runs out.

- Build fallback chains as ordered, pre-validated sequences. An untested fallback may fail under the same conditions that caused the primary to fail, turning a safety net into a second point of failure.
- Signal degradation explicitly when a fallback fires. Include a metadata flag in the output object so downstream components can adjust confidence, change routing, or refuse irreversible actions on partial data.

Programmatic Enforcement and Guardrails *(Lessons 25 and 26)*

Why prompt guardrails are probabilistic and where code-layer enforcement is the only architecturally sound answer.

- Prompt guardrails are behavioral guidance, not deterministic guarantees. Context drift in long runs, adversarial inputs, and edge cases outside training distribution can all produce non-compliant outputs even from a well-instructed model.
- Reserve prompt-only guardrails for low-stakes, reversible paths. Financial transactions, PII handling, and safety-critical actions require code-layer enforcement that runs regardless of model output.
- Pre-execution gates catch malformed inputs, out-of-range parameters, and unauthorized callers before any action is taken. Stopping a bad input at this layer costs almost nothing compared to reversing a bad action after the fact.
- Post-execution gates inspect output schema compliance, business rule adherence, and semantic anomalies before results are committed or forwarded downstream. Run both gate types in sequence on any high-stakes pipeline.
- Match enforcement response to actual risk level. Use hard blocks for non-negotiable safety and compliance constraints, soft warnings for advisory constraints where human review adds value, and escalation triggers for genuinely ambiguous cases.

Domain 2: Tool Design & MCP Integration

Exam weight: 18%

Section 2A - Tool Design Fundamentals

Tool Descriptions as the Primary Routing Mechanism *(Lessons 29 and 30)*

Why description quality, not tool names, determines which tool Claude calls and when.

- Claude routes by semantic matching against description text, not by tool name. A perfectly named tool still misroutes if its description is vague or incomplete.
- Four routing outcomes follow directly from description quality: correct call, misroute, no-call, and fallback to user clarification.
- Write every tool description with three parts: what it does, when to call it, and what it explicitly does not handle.
- Include input format hints and output shape in the description so Claude can both select the tool and form a correct call in a single step.
- Validate routing with positive, negative, and disambiguation tests before any toolset ships to production.

Tool Schema Design *(Lessons 31 and 32)*

How JSON Schema parameter choices directly shape the arguments Claude generates for every tool call.

- JSON Schema is the contract between Claude and your tool. Every property declared shapes argument generation at inference time, not just at definition time.
- Use specific, domain-meaningful parameter names in snake_case. Generic names like 'data' or 'input' leave Claude guessing at field purpose and increase argument errors.
- Treat parameter descriptions as micro-prompts. Omitting them degrades argument quality even when types are correct.
- Only mark a field required if the tool genuinely cannot function without it. Over-requiring fields forces Claude to hallucinate values to satisfy the schema.
- Use enum constraints to eliminate free-form string variation for fixed-value inputs. Declaring enum: ['low', 'medium', 'high'] makes routing fields deterministic and your backend logic simpler.

Tool Error Handling and Reliability *(Lessons 33 and 34)*

Designing tool errors and retry patterns that give Claude clear recovery paths instead of forcing it to guess.

- A well-structured error response has four fields: a stable error code, a human-readable message, a context block identifying what failed, and a suggested action for Claude.
- Distinguish user errors from system errors in your error codes. User errors need a correction from the caller; system errors like timeouts are candidates for retry without user involvement.

- Make state-changing tools idempotent by accepting a caller-generated idempotency key. This makes retry safe and prevents duplicate side effects on transient failures.
- Return a partial_success status with separate succeeded and failed lists for batch operations. A generic error hides what completed and forces Claude to re-run work that already succeeded.
- Apply the circuit breaker pattern when a downstream dependency has intermittent outages. It stops cascading calls, defines a recovery probe interval, and prevents the context window from filling with repeated errors.

Tool Distribution and Selection Across Agents *(Lessons 35 and 36)*

Why focused tool palettes and the specialist subagent pattern produce more reliable routing than large shared catalogs.

- Every tool description loads into context on every turn whether used or not. Tool bloat consumes tokens, slows inference, and introduces routing ambiguity by crowding out relevant content.
- The specialist subagent pattern assigns each agent only the tools its role requires. A retrieval subagent and a write subagent each carry a narrow palette, making correct selection the only reasonable outcome.
- Two-good-matches failure occurs when two tools both look like valid candidates. The model alternates between them non-deterministically. Fix it with distinct verbs and explicit scope boundaries, including what each tool does not do.
- Add explicit routing rules and tie-breaking instructions to the system prompt when descriptions alone cannot fully separate similar tools.
- Log every tool selection in production. Tracking call frequency surfaces selection drift before users notice it, and distinguishes routing degradation from genuine usage changes.

Section 2B - MCP Server Configuration

MCP Architecture and the Three Primitives *(Lessons 37 and 38)*

How MCP replaces fragmented custom adapters with one open protocol any compliant client and server can implement.

- MCP defines a uniform client-server interface so any AI model or runtime can connect to tools and data sources without custom glue code.
- The server hosts and exposes capabilities; the client connects, runs capability discovery, and routes requests, one client can hold multiple server connections at once.
- Three primitives cover every capability type: tools are model-controlled, resources are application-controlled, and prompts are user-controlled.
- Match the primitive to who drives the decision. Model triggers an action? Tool. Application loads background data? Resource. User selects a workflow starter? Prompt.
- MCP is model-agnostic by design. Servers don't know which model consumes them, so your infrastructure works with whatever AI runtime your team adopts next.

Building MCP Servers - Tools *(Lessons 39 and 40)*

How to set up an MCP server in Python, define tools with typed schemas, and verify them before connecting a real client.

- Every MCP server follows a three-step pattern: create the server instance, register tools before calling run(), then start the event loop and block.
- Register all tools before run() is called. Capabilities advertised during the initial handshake are locked in; anything added afterward is invisible to the client.
- Apply the @server.tool() decorator to register a Python function. The function name becomes the tool name; the docstring becomes the description the model reads to decide when to call it.
- Keep all tool handlers async. A synchronous I/O call inside a handler blocks the entire event loop and stalls every other in-flight request.
- Return a structured error dict with isError set to true when a handler fails. Structured errors let the model reason about failure without crashing the server or dropping client connections.

Resources and Prompts in MCP *(Lessons 41 and 42)*

How MCP resources and prompt templates are defined, who controls each one, and when server-side templates add real value.

- Every resource definition requires three components: a stable URI the client uses to address it, a MIME type declaring content format, and a read handler that returns the data.
- Static resources are safe to cache; dynamic resources are generated at request time and need subscriptions or explicit expiry to avoid injecting stale context into the model.
- Resources are application-controlled. The host application decides what to retrieve and inject; the model reads the injected content but did not select or request it.
- MCP prompt templates are parameterized instructions stored on a server that users explicitly select to pre-populate an interaction before it begins.
- Keep required template arguments minimal and provide defaults for optional ones. Each required argument adds friction, and high-friction templates see lower adoption regardless of how useful they are.

MCP Clients and Integration *(Lesson 43)*

How MCP clients manage the full connection lifecycle, discover server capabilities, invoke tools, and route across multiple servers.

- The MCP client is the mediator between the model and every tool server. The model never contacts a server directly; the client handles all connection management and routing.
- Use STDIO transport when the server runs locally as a child process. Use StreamableHTTP when the server is remote, shared, or needs to support multiple concurrent clients.
- Client initialization completes a three-phase handshake: connect, negotiate protocol version and capabilities, then confirm with an initialized notification before any tool is invoked.

Section 2C - Advanced MCP and Built-in Tools

MCP Transport Mechanisms *(Lessons 44 and 45)*

How STDIO and StreamableHTTP differ, and the one question that decides which transport belongs in any deployment.

- STDIO spawns the server as a child process and routes all messages over stdin and stdout. Zero network configuration is required.
- STDIO is stateful by design. In-memory state persists across all tool calls within a session because the process stays alive.
- Claude Code uses STDIO as its native MCP transport. Each registered server runs as a child process spawned from the settings file.
- StreamableHTTP turns the server into an HTTP service. It supports request-response for standard calls and SSE streaming for long-running operations.
- Apply one decision rule: if a deployment crosses a network boundary or serves more than one user, STDIO is eliminated and StreamableHTTP is the answer.

Advanced MCP Features *(Lessons 46 and 47)*

Sampling, notifications, and roots are the three features that extend MCP servers beyond basic request-response into production-grade intelligence.

- Sampling lets a server request an LLM inference call through the client. Model access, credentials, and policy controls stay with the client owner, not the server.
- Implement a sampling handler by registering an async callback on the client. The SDK routes all incoming createMessage requests to that handler automatically.
- Progress notifications carry a numeric progress value and a progressToken tied to the originating tool call. Use them to drive UI feedback during long operations.
- Log notifications stream diagnostic messages with severity levels during execution. Use log for observability, progress for user-facing status indicators.
- Roots are an explicit grant from client to server, listing filesystem paths the server may access. Servers have no implicit access to any path without a roots list.

MCP Security and Production Considerations *(Lessons 48 and 49)*

Every MCP tool definition is a trust boundary, and every production deployment needs authentication, versioning, and observability to stay safe.

- Apply least privilege at design time. Give each tool only the access its specific function requires, and separate tools for separate resource types.
- Enforce access through three layers in combination: schema constraints reject out-of-range inputs, roots limit filesystem scope, and server-side auth checks caller identity.
- Sanitize every argument at the server boundary before any handler logic runs. Treat inputs as untrusted regardless of where the client claims they originated.
- OAuth 2.0 is the MCP-specified authentication mechanism for production StreamableHTTP servers. Token scopes can map directly to which tools a client may invoke.
- Rate limiting and health endpoints are non-optional for shared servers. Return HTTP 429 with a Retry-After header on limit hits, and enforce hard timeouts on handler execution.

Claude *(Lessons 50 and 51)*

Claude ships five first-party built-in tools that require no schema work, each with a defined scope, cost profile, and capability ceiling architects must know.

- The five built-ins are web_search, computer_use, code_execution, file tools, and Bash. Enable each via a typed parameter rather than a custom schema.
- web_search constructs queries internally, returns ranked snippets with source citations, and is bounded by provider rate limits. You cannot pass a raw query string directly.
- computer_use operates inside a sandboxed environment. Screenshot, mouse, and keyboard primitives cannot reach the host operating system or any private network.
- code_execution runs in an ephemeral container that is discarded after the session. State does not persist across calls, so all logic must be stateless by design.
- Reach for custom tools when the workflow involves internal APIs, regulated data, domain-specific schemas, or audit trail requirements that built-ins cannot satisfy.

Domain 3: Claude Code Configuration & Workflows

Exam weight: 20%

Section 3A - Claude Code Fundamentals

Claude Code Architecture *(Lessons 52 and 53)*

What Claude Code actually is, how the planning loop runs, and what the permission model controls before any tool fires.

- Claude Code is an autonomous agentic CLI, not a chat interface or autocomplete plugin. It executes multi-step tasks end to end.
- The planning loop runs four phases continuously: plan, tool call, observe, repeat. It keeps cycling until the task is complete.
- Distinguish Edit from Write. Edit makes targeted find-and-replace changes; Write overwrites the entire file and destroys what it doesn't include.
- Bash is the most capable and most dangerous tool in the set. Scope it carefully with permissions, because there is no undo for a destructive shell command.
- Permissions are enforced before any tool runs and are separate from hooks. If a permission blocks an action, no tool call happens and no hook fires.

Project Setup and Context *(Lessons 54 and 55)*

How to onboard Claude Code to a real codebase and keep context accurate, concise, and working across every session.

- Run the claude command from the project root. That directory sets the boundary for what Claude Code discovers automatically.
- Use /init to scaffold a starter CLAUDE.md from repo analysis. Always review and refine the output before team adoption, it is a draft, not a finished configuration.
- Include context that changes what Claude Code would do: architecture decisions, coding conventions, known constraints. Omit anything whose absence wouldn't affect the output.
- Configure .claudeignore to exclude secrets, large binaries, and auto-generated files. It serves as both a security control and a context efficiency tool.
- CLAUDE.md is standing context loaded at every session start, not a conversation log. Claude Code has no persistent memory across sessions without it.

Section 3B - CLAUDE.md Configuration Hierarchy

CLAUDE.md Hierarchy *(Lessons 56 and 57)*

The three-level CLAUDE.md hierarchy and how precedence rules keep team standards from being overridden by personal preferences.

- CLAUDE.md has three levels: user at ~/.claude/CLAUDE.md, project at the repo root, and team distributed via shared or symlinked files across projects.
- All three levels load simultaneously at session start. Precedence only activates when instructions directly conflict; most combinations are additive.
- Project-level overrides user-level on conflicts because project config is version-controlled and applies to every contributor, not just one developer.
- Keep personal style preferences at user-level and team standards at project-level. Putting team rules in user-level config breaks consistency silently for other contributors.
- Treat CLAUDE.md as a static configuration file, not a memory store. Every token it contains counts against the context window for that session.

Path-Specific Rules and Scoping *(Lessons 58 and 59)*

Subdirectory CLAUDE.md files and .claudeignore give you precise control over which rules fire where and which files Claude Code can see.

- Place a CLAUDE.md inside any subfolder and Claude Code loads it automatically when working in that path. No registration or manifest entry required.
- Subdirectory configs augment the parent config, they do not replace it. Parent instructions stay active; the subdirectory file adds folder-specific rules on top.
- Use directory scoping for framework-specific rules in /frontend, never-delete constraints in /infrastructure, and a single no-edit directive in /generated folders.
- .claudeignore uses gitignore syntax to exclude files from context. Always exclude .env files, credentials, and customer data to prevent accidental exposure.
- Performance exclusions like node_modules and .git free context space that would otherwise be consumed by irrelevant content. Both security and performance are valid reasons to add entries.

Writing Effective CLAUDE.md Instructions *(Lessons 60 and 61)*

Effective CLAUDE.md directives are specific, actionable, and unambiguous, and they encode only what Claude Code cannot already infer from training.

- An effective directive names the exact version, path, or convention. Vague instructions may be read but produce inconsistent behavior across sessions.
- Write CLAUDE.md like a style guide checklist, not a design document. Short bullet points per rule outperform paragraphs buried in narrative prose.
- Encode repo-specific decisions: runtime version pins, naming conventions, test coverage thresholds, PR process, and security rules like no hardcoded credentials.
- Skip general programming knowledge Claude Code already has. Every line explaining standard library behavior or common patterns wastes context space that could hold actual code.
- Avoid duplicating rules that linters or formatters already enforce. Let tools cover what tools can cover, and reserve CLAUDE.md for the gaps automated tooling cannot fill.

Section 3C - Commands, Skills, and Subagents

Custom Slash Commands *(Lessons 62 and 63)*

How to build, name, and distribute reusable slash commands that the whole team gets automatically.

- A slash command is a Markdown file in .claude/commands/ or ~/.claude/commands/ whose filename becomes the /keyword with no extra registration required.
- Project-level commands live inside the repo and distribute through version control; every developer who clones the project gets them automatically.
- User-level commands stay in ~/.claude/commands/ on your machine and follow you across projects but are never shared.
- Drop $ARGUMENTS into a command file where variable input should land; text typed after the keyword replaces the placeholder at invocation time.
- Follow the verb-noun naming convention and keep each command to a single clearly defined task so the library stays predictable and self-documenting.

Skills: Reusable Agent Instructions *(Lessons 64 and 65)*

Skills are trigger-activated instruction templates that fire automatically when a condition is met, without the user typing anything.

- Skills occupy the middle layer of the instruction stack: more selective than always-on CLAUDE.md rules and more passive than autonomous subagents.
- A SKILL.md file requires three frontmatter fields to be recognized: name, description, and trigger. Missing any one causes silent failure with no error.
- Phrase-based triggers fire when user input matches the pattern; event-based triggers fire on system actions like a build completing or a file being created.
- Evaluate whether to build a skill on three factors: frequency across sessions, complexity of the workflow, and reusability across team members.
- Personal skills live in ~/.claude/skills/, project skills in .claude/skills/, and enterprise skills are pushed via managed settings to the entire organization.

Subagents in Claude Code *(Lessons 66 and 67)*

Subagents are separate Claude Code sessions with isolated context, configured tools, and scoped permissions for delegated work.

- Spawn a subagent via the /agents command interactively or via the Agent tool programmatically inside a running agentic workflow.
- Each subagent starts with a completely empty context window; nothing from the parent transfers automatically, so every piece of needed context must be passed explicitly.
- Configure tool access, directory scope, and permissions per subagent independently of the parent, applying the principle of least privilege to each delegated task.
- Answer three design questions before spawning: what is the exact task, what context does the subagent need, and what structured output format should it return.

- Reserve delegation for work that is truly independent or benefits from isolation; inline execution is better when the task is simple or requires full parent context.

Plan Mode and Iterative Refinement Workflows *(Lessons 68 and 69)*

Plan mode keeps thinking and execution separate so you can review and correct Claude's approach before any file is written.

- Trigger plan mode explicitly with /plan before a request; Claude reads relevant files, produces an ordered proposal with files and risks called out, and waits.
- The approval gate is the primary control point: accept the plan, edit it inline to trim scope or reorder steps, or reject it and restate your intent.
- Plan mode pays off most on cross-file refactors, schema changes, and ambiguous requests where catching a wrong assumption before execution saves significant rework.
- When reality diverges mid-execution, stop and re-plan rather than letting Claude improvise silently around an assumption that no longer holds.
- Keep plans to five to eight numbered steps with explicitly named files and stated risks so they are easy to review, edit, and approve reliably.

Section 3D - Hooks, SDK, and CI/CD

Hooks - Lifecycle Events and Implementation *(Lessons 70 and 71)*

The four hook types, how exit codes give you deterministic control, and the implementation gotchas that silently break sessions.

- Hooks are shell-level scripts registered in Claude Code settings, not in CLAUDE.md. They fire at lifecycle events regardless of what Claude decides to do.
- PreToolUse is the only hook that can block an action. It fires before the tool runs, and a non-zero exit code stops execution immediately.
- PostToolUse fires after a tool call completes. Use it to log results or trigger side effects, but never to prevent an action already taken.
- Exit code 0 means continue; any non-zero exit means block. A hook intended to warn silently will silently block every matching tool call if it returns non-zero.
- Test every exit code path in isolation before deploying a hook. A misconfigured PreToolUse hook looks identical to Claude refusing to act.

Useful Hook Patterns *(Lesson 72)*

Six practical hook patterns map common automation and safety needs to the right lifecycle event.

- Register a PostToolUse hook on file writes to run a code formatter automatically. Every saved file stays style-compliant without any manual step.
- Use PreToolUse on the Bash tool to inspect command strings for destructive patterns like rm -rf. A non-zero exit halts the command before it executes.
- PostToolUse logging records what succeeded and what was produced. PreToolUse logging records what was attempted, including actions that were later blocked.

Claude Code SDK *(Lesson 73)*

The Claude Code SDK replaces the terminal with method calls, giving your code full programmatic control over agentic sessions.

- The SDK and CLI expose the same capabilities. The difference is who drives the session: a human at a terminal, or your code calling SDK methods.
- Use the SDK for agentic sessions requiring multi-step tool use, file edits, or shell commands. Use the direct API for single-turn model completions.
- Instantiate a session object with the model, tool permissions, and working directory. Pass prompts as plain strings, no terminal interaction is required.

CI/CD Integration and Automation *(Lesson 74)*

The -p flag is the one thing standing between Claude Code and a pipeline job that hangs indefinitely waiting for input that never arrives.

- Without -p, Claude Code enters interactive mode in a CI/CD runner where no terminal exists. The job blocks until it times out.
- Pass -p (or --print) with your prompt to activate non-interactive mode. Claude Code executes the task, writes output to stdout, and exits with a status code.
- Keep each pipeline invocation atomic and write self-contained prompts. The agent has no memory of prior runs unless you include that context explicitly.

Domain 4: Prompt Engineering & Structured Output

Exam weight: 20%

Section 4A - Core Prompt Engineering

Writing Clear and Specific Instructions *(Lessons 75 and 76)*

Specificity is the single most reliable lever you control when writing prompt instructions.

- Vague instructions do not stop the model. They force it to choose an interpretation, silently, and proceed with the assumption you may not have intended.
- Every added constraint narrows the output space. Format, length, audience, and perspective each close a different category of interpretation gap.
- Use the direct instruction principle: state what you want the model to do, not what to avoid. Negative framing leaves the positive goal undefined.
- Decompose complex instructions into ordered steps, each with one clear goal. Sequential steps are independently verifiable and far easier to debug.
- Test any instruction with the interpretation test and the gap test before running it. Two plausible outputs means you need one more constraint.

System Prompts and Role Definition *(Lessons 77 and 78)*

The system prompt sets the foundation every model response is built on for the entire session.

- Separate stable content from request-specific content. Role, standing rules, and domain knowledge belong in the system prompt; dynamic data belongs in the user turn.
- Structure the system prompt in five ordered sections: role, context, instructions, constraints, and examples. Lead with role because it frames everything that follows.
- Role assignment changes vocabulary, reasoning depth, and tone throughout a session. It is the fastest lever for aligning model behavior to a specific use case.
- Persona extends role with a named identity and explicit stylistic constraints. Specificity in persona design produces stability; vague descriptions produce inconsistent behavior.
- Audit system prompts for conflicting rules before deploying. Test across adversarial inputs and topic switches to surface failures before they reach production.

XML Tags and Document Structure *(Lessons 79 and 80)*

XML tags give Claude structural signals that plain text delimiters cannot reliably provide.

- Claude's training reinforces XML tag boundaries over prose separators. Use the four standard tags: instructions, context, examples, and output_format.

- Separate task directives from user-supplied data using XML tags. Content inside a context tag cannot override content inside an instructions tag, which is your primary defense against prompt injection.
- Keep tag naming descriptive and consistent. Every tag needs a matching close, and names should align across all prompts in a system to maintain structural signal at scale.
- Use nested tags when a section has internal structure the model must parse separately. One or two levels of nesting is the practical limit before readability degrades.
- For multi-document prompts, wrap each source in a document tag with numbered attributes and explicit source metadata. Reference documents by index or name in instructions to prevent output misattribution.

Section 4B - Advanced Prompting Techniques

Few-Shot Prompting and Example Design *(Lessons 81 and 82)*

How examples anchor model output more reliably than prose instructions, and how to design them well.

- A single example simultaneously communicates format, tone, reasoning pattern, and scope, things prose instructions address one dimension at a time.
- Models extract implicit rules from demonstrated output through pattern matching, eliminating the interpretation step that prose instructions require.
- Format anchoring locks output structure from the very first token because the model continues directly from the format the example ends in.
- Three examples is the most broadly useful shot count for structured tasks; beyond three, returns diminish unless inputs span genuinely different types.
- Audit example quality before adding more examples, one strong, artifact-free pair outperforms five inconsistent ones that send conflicting signals.

Chain-of-Thought and Extended Thinking *(Lessons 83 and 84)*

Two distinct techniques for improving reasoning quality, one a prompt-text tool, the other an API feature.

- Chain-of-thought prompting lives entirely in the prompt text, append 'Think step by step' to trigger visible reasoning without any API parameter changes.
- Zero-shot CoT requires only an elicitation phrase; structured CoT adds a labeled scaffold for consistent reasoning across similar tasks.
- CoT helps most on multi-step arithmetic, logic, planning, and complex instruction following; simple factual lookups gain nothing from it.
- Extended thinking is an API feature that sets a token budget for internal reasoning blocks streamed before the final response, not part of the output text.
- Thinking blocks count against the context window and require client-side streaming event handling, try chain-of-thought first before adding that complexity.

Prefilling and Output Steering [REWORK GAP - deck 86 removed] *(Lessons 85 (deck 86 REMOVED 2026-07-30))*

[REWORK GAP] Output steering without prefilling (prefill is deprecated). Deck 86 removed and unpublished; re-author before rebuild.

- [DEPRECATED] Prefilling places a partial assistant-role message at the end of the messages array, committing the opening token and locking downstream format before generation starts.
- [DEPRECATED] Best prefill use cases are JSON openers, XML root tags, format anchors, and language starters, appropriate use is always about format, not bypassing refusals.
- [DEPRECATED] A format lock pairs a few-shot example in the user turn with a prefill opener in the assistant turn, giving two reinforcing signals for maximum structure reliability.
- Framing shapes tone and scope before the model reaches any explicit instruction, audience, perspective, and scope all steer output independently of directives.
- Over-steering through conflicting or redundant constraints degrades quality; layer framing and format instructions to target distinct dimensions ([DEPRECATED: drop 'prefilling' as a layer - deck 86]).

Temperature, Sampling, and Generation Parameters *(Lesson 87)*

What temperature, top_p, and top_k actually do to token selection, and when to adjust them.

- Temperature scales logit probabilities before sampling, lower values concentrate output on high-probability tokens, higher values spread it across more candidates.
- Use low temperature for classification, extraction, and factual tasks; use higher temperature for creative writing and brainstorming where output diversity is the goal.
- Temperature zero is not a universal safe default, it produces flat, repetitive output on creative tasks and mechanical responses in conversational contexts.

Multi-Turn Conversation Design *(Lessons 88 and 89)*

When dialogue outperforms a single prompt, how to design each turn, and how to manage context over time.

- Multi-turn dialogue wins when tasks are ambiguous, outputs are interdependent, a human must approve mid-way, or task length creates context window pressure in a single call.
- A good clarification turn acknowledges what was understood, asks exactly one targeted question about the highest-risk ambiguity, and sets an expectation for what comes next.
- Turn sequences assign one clear goal per turn and pass the model's prior output as explicit input to the next step, keeping each exchange small and reviewable.
- Context drift is a gradual failure where topic, tone, or task framing shifts across turns without any single obvious error, periodic re-anchoring prevents it from compounding.
- Reset a conversation rather than continuing when noise buildup or drift has made accumulated history a liability; carry forward only key decisions and confirmed constraints.

Section 4C - Structured Output and Evaluation

JSON Schemas and Structured Data *(Lessons 90 and 91)*

How JSON mode and schema-constrained output differ, and which mechanism to reach for based on strictness requirements.

- JSON mode guarantees parseable syntax at the token level, not schema compliance. Field names, types, and required keys are entirely up to the model.
- Use the reliability hierarchy to pick your mechanism. Tool use enforces schemas structurally; JSON mode enforces syntax only; format instructions and few-shot examples fall below both.
- When field compliance is non-negotiable, tool use is the right choice. The model is trained to honor declared parameter schemas, which JSON mode cannot replicate.
- JSON mode is appropriate when your downstream system tolerates flexible field structure or validates schema after parsing, and when no tool support is available.
- Schema quality is an active part of extraction reliability. Field descriptions, enum constraints, and correct required versus optional designations all affect how consistently Claude fills fields.

Tool Use for Structured Output *(Lessons 92 and 93)*

Tool definitions as extraction schemas, multi-tool composition, and the validation loop that catches what even good schemas miss.

- Treat tool use as a pure output mechanism, not only a side-effect trigger. The model emits a structured argument object that conforms to the declared parameter schema without needing a parsing step.
- Design extraction schemas with precise field descriptions. A field named without context leaves the model to guess; a description that names boundary cases and expected format drives consistent output.
- Compose multiple focused tool definitions instead of one large schema. Each tool stays small, each call is independently validated, and the error surface shrinks.
- Build validation loops that run after every extraction call. Validate schema compliance, business rules, and semantic correctness in layers, then re-prompt with the specific error on failure.
- Ground extraction calls with the source document and ask for citations. Grounding converts generation from memory-based to evidence-based and makes unsupported extractions auditable.

Validation-Retry Loops and Multi-Pass Review *(Lessons 94 and 95)*

Validation gates, retry budgets, and the reviewer pattern that catches content failures before the user sees them.

- Run three distinct validation layers in sequence. Schema validation checks structure; format validation checks field-level constraints; semantic validation checks whether the content actually means what it should.
- Include the specific failure reason in every retry prompt. A vague instruction to try again gives the model nothing to act on and reliably reproduces the original failure.
- Set a hard retry budget of two to three attempts and escalate explicitly when it is exhausted. Returning silence is the most dangerous failure mode in a validation loop.
- When a model gets stuck repeating the same failure, add a small temperature bump and vary the prompt slightly on each retry to break the pattern rather than repeating it louder.
- Use the reviewer pattern for high-stakes output. A second model call critiques the first against an explicit checklist before delivery, and a revision pass closes the loop with a concrete improvement.

Prompt Evaluation and Testing *(Lessons 96 and 97)*

Building a systematic eval workflow that replaces spot-checking impressions with measurable, repeatable deployment evidence.

- Manual spot-checking cannot catch edge cases, adversarial inputs, or regressions. A systematic eval workflow produces documented, repeatable evidence that a change is safe to deploy.
- The core eval loop runs four steps every iteration: build a dataset with expected outputs, run the prompt against each case, grade every response, then compare pass rate and regression rate against the baseline.
- Track four metrics to evaluate a prompt change: pass rate, regression rate, latency per call, and cost per eval. Together they cover quality, stability, speed, and budget.
- Run the full eval suite after every prompt change, not just against related cases. A fix that introduces regressions elsewhere is not a net improvement regardless of intent.
- Choose the grading method to match the output type. Code-based grading is fast and deterministic for structured output; model-based grading scales to subjective tasks but requires calibration against human judgments before the scores can be trusted.

Domain 5: Context Management & Reliability

Exam weight: 15%

Section 5A - Context Preservation

Context Window Architecture and Limits *(Lessons 98 and 99)*

How tokens, context components, and model-tier limits define the fundamental constraint on every Claude inference call.

- The context window holds everything the model can see in a single call: system prompt, conversation history, retrieved documents, tool schemas, and the model's own output.
- Tokens are the atomic unit of measure, roughly four characters each; input and output tokens both count against the limit and are billed at separate per-token rates.
- Match your model tier to the task's context requirements. Haiku suits short tasks, Sonnet handles multi-document retrieval, and Opus covers full-codebase or long-horizon agentic sessions.
- Avoid filling the context window without a strategy. Near-full windows cause hard truncation, request failures, or the lost-in-the-middle effect where mid-window content receives weaker model attention.
- Define a context budget in code by allocating token limits across system prompt, history, retrieved chunks, and output headroom before a single request is sent.

Fact Extraction into Persistent Blocks *(Lessons 100 and 101)*

Why history summarization silently drops critical facts and how the extract-first pattern keeps them addressable across the full session.

- History summarization is a fluency-for-specificity tradeoff. The compressed narrative reads cleanly but drops identifiers, numeric limits, named entities, and committed decisions without warning.
- Loss compounds with every summarization pass. A constraint stated at turn five may be completely gone by turn twenty as each pass compresses an already-compressed summary.
- Run fact extraction before summarization. Pulling structured facts into a persistent block first means the compressor can never discard them, breaking the loss cycle entirely.
- Choose append-only when fact history matters and replace-on-conflict when only the current value is needed. Cap block size with aging, merging, or priority tiers to stay within your token budget.
- Always include an explicit instruction to consult the persistent block in every request. A fact block with no instruction pointing to it is an expensive no-op.

Trimming Verbose Tool Outputs to Relevant Fields *(Lessons 102 and 103)*

How unchecked tool-output bloat erodes agent accuracy, compounds across multi-call workflows, and which trim strategy fits each scenario.

- APIs return far more fields than any single task requires. Every extra field is a token that raises per-request cost, increases latency, and competes with relevant signal for model attention.

- Bloat compounds across multi-call workflows. After five chained tool calls, the context holds five layers of noise that later reasoning steps must attend to before reaching the actual signal.
- Use field-level filtering when the tool schema is stable: declare an allowlist at the wrapper, run deterministic JSON extraction, and remove noise before it ever enters context.
- Use model-driven summarization when the schema varies across providers or versions. A lightweight model call adapts to any response structure and returns only what the main agent needs.
- Log every field the trim layer discards. A trim audit log is the first place to check when a downstream step fails, and it reveals over time when your allowlist has grown too narrow.

Section 5B - Escalation and Error Propagation

Escalation Design: Explicit Triggers and Structured Handoff Protocols *(Lessons 104 and 105)*

How to design explicit escalation triggers and handoff payloads so agents hand off cleanly rather than silently failing.

- Escalation fails in two directions: agents that never escalate produce silent bad output, while agents that escalate constantly train reviewers to ignore alerts.
- Hard triggers enforce policy without model judgment, a denylist hit always fires; soft triggers are tunable confidence or intent checks the model weighs.
- Define an ask-for-help tool in the system prompt so the model can escalate deliberately, producing a clean structured record instead of a free-text message.
- A handoff payload needs four fields: conversation summary, triggering question verbatim, attempted actions with results, and a suggested next step from the agent.
- Close the loop by re-injecting the reviewer's resolution back into the original agent's context; without that step, escalation is a one-way street.

Error Propagation in Multi-Agent Systems *(Lessons 106 and 107)*

Where errors belong in a multi-agent stack, and how to design a propagation contract that prevents both silent corruption and coordinator thrash.

- Local recovery is correct when the coordinator's plan would not change; propagate scope violations, policy denials, and exhausted retry budgets up the stack.
- Use the coordinator-needs-to-know test: if the failure affects scope, sequencing, or resource allocation, surface it with a structured error envelope.
- Classify errors as retryable or non-retryable before applying any local retry budget; repeating a non-retryable call locally wastes cycles and delays the coordinator's response.
- The error envelope carries typed fields, error_type, severity, context, and a recovery_hint, so the coordinator can branch deterministically without parsing free-text strings.
- Include a completed_steps field in the envelope when subagents fail mid-task; the coordinator uses it to resume from the failure point rather than re-executing finished work.

Distinguishing Access Failures from Empty Results *(Lessons 108 and 109)*

Why an empty tool result can mean two different things, and the wrapper discipline that prevents agents from confusing the two.

- An access failure means the tool never reached the data source; an empty result means it did and confirmed no match exists, agents cannot tell them apart without explicit signaling.
- Agents default to reporting 'found nothing' on both outcomes because an empty list with no error flag triggers no failure-handling branch in the model's reasoning.
- Build a tool wrapper that maps HTTP outcomes to a typed status enum before the agent sees the response: success, empty_result, partial_result, or access_failure.
- Retry policy follows the status field, not the data field: access failures retry with exponential backoff up to a configured limit; empty results never retry.
- Reserve 'found nothing' for confirmed empty results only; say 'could not check' on access failures so users never mistake a tooling gap for a genuine absence of data.

Section 5C - Context Efficiency for Large Codebases

Scratchpad Files for Cross-Session Knowledge Persistence *(Lessons 110 and 111)*

How a plain text file in the repo carries decisions, dead ends, and task state across session resets.

- Every new Claude Code session opens with a blank context window. Without a persistence mechanism, each session repeats the same costly discovery work.
- Store architectural decisions and their rationale, dead ends already explored, naming conventions, and the current task state block in the scratchpad.
- Never put code, credentials, or private user data in the scratchpad. Code drifts from the real source, credentials are a security violation, and sensitive data has no place in a committed file.
- List the scratchpad as one of the first files to read in CLAUDE.md so the agent loads it reliably at session start, not optionally.
- Update the scratchpad before closing every session. A stale scratchpad misleads the agent and is worse than having no scratchpad at all.

Subagent Delegation to Isolate Verbose Discovery *(Lessons 112 and 113)*

How delegating multi-file investigation to a child agent keeps the main agent's context clean and focused.

- Discovery across a large codebase is inherently verbose. Inline discovery forces every grep result and dead-end file read into the main agent's context window.
- Delegate discovery by having the subagent investigate and return only a structured summary. Verbose intermediate state lives in the subagent's disposable context, never the parent's.
- Write the subagent prompt as a precise question with explicit scope, return shape, and a stop condition. Vague prompts produce wall-of-text returns that re-pollute parent context.
- Give the subagent read and search tools only. Write and delete tools have no place on a discovery palette and introduce unintended side effects.
- Treat subagent output as a hypothesis, not verified truth. Spot-check critical findings before acting on them in high-stakes downstream work.

Long-Conversation Coherence, Compaction, and Session Memory *(Lesson 114)*

How to manage token accumulation, compress conversation history, and prevent goal drift in extended multi-turn sessions.

- Every turn in a multi-turn conversation appends to history. When the window fills, providers truncate older turns, erasing earlier instructions and established facts.
- Compaction replaces verbose prior turns with a shorter summary, reducing token load while preserving the essential thread. It is lossy by design, so avoid it where verbatim history is required.
- In-context memory is simple but window-bounded. External memory retrieves stored summaries or facts at turn start and scales to multi-session use cases.

Section 5D - Human Oversight, Confidence, and Provenance

Field-Level Confidence Scores and Calibrated Validation Sets *(Lessons 115 and 116)*

Why a single aggregate confidence score hides which fields are wrong, and how calibration makes per-field scores trustworthy.

- A record-level score masks weak fields. A date field sitting at 0.40 stays invisible behind an overall score of 0.85.
- Elicit per-field confidence through schema design. Each field object carries a value key and a confidence key, populated in a single structured output call.
- Route on the field, not the record. High-confidence fields auto-accept downstream while only uncertain fields enter the human review queue.
- Model-emitted scores are not automatically trustworthy. A model can report 0.90 on fields where it is actually correct only 60% of the time.
- Calibrate against a held-out labeled validation set before setting production thresholds. Re-calibrate after every model upgrade or prompt change.

Stratified Sampling and Routing Low-Confidence Output to Human Review *(Lessons 117 and 118)*

How to concentrate reviewer attention on the outputs most likely to be wrong, and how a review queue turns that signal into auditable action.

- At 95% model accuracy, a random review sample contains 95 correct outputs and only 5 errors. Reviewers spend most of their time confirming what the model already knows.
- Stratify review selection across three dimensions: confidence score, output category balance, and downstream stakes. Combining all three maximizes errors caught per reviewer-hour.
- Set per-category sample quotas to protect against class imbalance. Rare categories with high error rates are underrepresented in any purely random review queue.
- Design the review queue with a single action surface. Reviewers approve, edit, or reject without switching tools, and every action generates a logged record.
- Reviewer edits create labeled correction pairs for future calibration. Reviewer rejects flag input classes the model handles poorly. Every action is training signal.

Provenance Tracking *(Lessons 119 and 120)*

How to make synthesized output auditable by attaching source pointers to every claim, and how to handle conflicts when sources disagree.

- Attach an inline source pointer to every claim in synthesized output. End-of-paragraph citation lists tell you which sources exist but not which sentence came from which source.
- A claim-to-source schema stores claim text paired with source identifiers and an optional per-source confidence score, making each claim a first-class auditable object.
- Run a sampling audit on 10 to 15 percent of claims to catch fabricated citations. A model can produce a real-looking source reference that does not support the claim or does not exist.
- When two sources conflict, apply an explicit resolution strategy: prefer-newer, prefer-authoritative, or escalate-to-human. Never silently average conflicting claims.
- Log every detected conflict even after it is resolved. Future reviewers need to know a choice was made, not that consensus existed.

Keep Going

This study pack is the companion to the full course. The course itself goes deeper on every topic here, with video lessons, practice exams written to the real exam blueprint, and detailed answer explanations for every question.

Everything lives at

www.certmasterclass.com

Create a free account and every cert's study pack comes with it.

Scan or tap to go straight to Claude Certified Architect.